Step 3: Download the DevStack master branch using the Git clone command from the link *http://git.OpenStack.org/OpenStack-dev/devstack*. The output after executing this command is shown below:

```
# git clone https://git.openstack.org/openstack-dev/devstack
```

It will take a few minutes to complete the cloning and this will depend upon your connectivity speed.

Step 4: Now, change the directory to the *devstack* directory *(# cd devstack/).* Use the *ls* command to make sure that the folders listed below and the files are in the *devstack* directory.

```
# cd devstack/
# ls
```

Step 5: Once we are in the *devstack* directory, go to the *Tools* directory and execute the script *create-stack-user.sh*. This script execution will create a *stack* user and we have to provide *sudo* privileges for this user.

```
# cd tools/
# ./create-stack-user.sh
```

Step 6: This is an optional step. But it would be better to edit the 'sudoers' file as shown in the command below.

```
# vi /etc/sudoers
```

Step 7: Change the ownership of the *devstack* directory using the *chown* command, where we downloaded the DevStack files. This is the best practice to avoid permission issues while executing the *stack.sh* script later.

```
#chown -R stack:stack /devstack/
```

Step 8: Now run the *stack.sh* script. Before that, we need to switch to the *stack* user and create an empty file called *localrc* in the *devstack* directory. Before creating the *localrc* file, make sure that you are in the cloned *devstack* directory.

```
#su stack
# vi localrc
```

The *localrc* file will contain all the passwords which we enter while running the *stack.sh* script. Later, if we execute the *stack.sh* script, there's no need to enter all the passwords, as *localrc* holds them already.

Note: *stack.sh* can be run only by a non-root account, which is why we had to create a stack user.

Now execute the *stack.sh* script and enter the passwords when prompted. It will take some time to complete the installation. After successful installation, we can see the output shown below:

```
# ./stack.sh
```

Running Tempest on DevStack

DevStack contains the inbuilt Tempest suite. The required configuration file can be seen in the */opt/stack/tempest* directory.

To run a unit test, change the directory to */opt/stack/tempest* and execute the *run_tempest.sh* script.

```
# ./run_tempest.sh
```

This will create *venv* and start the execution of unit tests.

```
# tox -efull
# tox -esmoke
```

Key findings

Here is a list of the key findings of all that we have

Key Findings-Failed Tests analysis — Kilo

OS/Tests	toxefull	toxesmoke	run_tempest
CentOS 7.0	Most of the Tempest test case failures are • Volume APIs • Third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) ReST API failures	Most of the Tempest test case failures are • Volume APIs related	Most of the Tempest test case failures are • Scenario tests, snapshot pattern related, Volume APIs and third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS)ReST API failures.
Ubuntu 14.04	Most of the Tempest test case failures are • Compute scenario and Cinder volume related Third party bots (this is related to Elastic Compute (EC2) and Object storage S3 features of AWS) ReST API failures.	Most of the Tempest test case failures are • Compute, scenario and Volume Rest API failures	Most of the Tempest test case failures are • Compute, scenario, Volume and third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) ReST API features.
Fedora 21	Most of the Tempest test case failures are • Volume APIs • Third part bots (this is related to Elastic Compute (EC2) and Object storage S3 features of AWS) ReST API features.	• Only one of the test case failed here and is related to Volume ReST API failure.	Most of the Tempest test case failures are • Volume APIs and third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) ReST API failures.

Figure 8: Key findings for Kilo release

Key Findings-Failed Tests analysis — Liberty

OS/Tests	toxefull	toxesmoke	run_tempest
CentOS 7.0	Most of the Tempest test case failures are • Volume APIs • Third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) • Server multi-node ReST API failures.	Tempest test case failure is related to following • Volume APIs	Most of the Tempest test case failures are • Compute, Volume APIs, basic-Ops test server, multi-node • third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) ReST API failures
Ubuntu 14.04	Most of the Tempest test case failures are • third party bots (this is related to Elastic Compute (EC2) and Object storage S3 features of AWS) ReST API failures	No failures found	Most of the Tempest test case failures are • Compute, scenario, Volume and third party bots (this is related to Elastic Compute (EC2) and Object storage S3 features of AWS) ReST API failures.
Fedora 21	Most of the Tempest test case failures are • scenario tests, compute, snapshot pattern, Volume APIs and third party bots (this is related to Elastic Compute (EC2) and object storage S3 features of AWS) ReST API failures	Found two failures which are • Volume ReST API failure	Most of the Tempest test case failures are • Volume APIs and third party bots (this is related to Elastic Compute (EC2) and Object storage-S3 features of AWS) ReST API failures.

Figure 9: Key findings for Liberty release